

กระบวนวิชา 261453 Digital Image Processing
รายงานวิชา 261453

จัดทำโดย

นาย พีริชญ์ สมัย
รหัสนักศึกษา 650610876

เสนอ
รศ.ดร.ศันสนีย์ เอื้อพันธ์วิริยะกุล

รายงานนี้เป็นส่วนหนึ่งของวิชา 261453
ภาคเรียนที่ 2/2568

มหาวิทยาลัยเชียงใหม่

คำนำ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 261453 Digital Image Processing โดยในส่วนแรกนั้นมีวัตถุประสงค์เพื่อศึกษาการประมวลผลภาพใน spatial domain และ frequency domain ผ่าน fourier transform ใน Digital Domain ผ่านการดำเนินการต่างๆไม่ว่าจะเป็นการทำ padding หรือการคูณพารามิเตอร์บางค่าเข้าไป สังเกตผลของเฟส และ ความถี่ และเปรียบเทียบผลที่ได้จากการดำเนินการที่ต่างกัน จากกระบวนการที่ต่างกัน ในส่วนที่สองนั้นจะเป็นการทำตัวกรองรูปภาพในโดเมนดิจิทัลเพื่อศึกษาผลของความถี่ใน system และผลของ cut off frequency ที่ต่างกัน

ในส่วนการพัฒนาโค้ดทั้งหมด ดำเนินการด้วย octave โดยในบางข้อนั้นไม่มีการเรียกใช้ไลบรารีภายนอก ยกเว้นในส่วนของการแปลง Toolbox หรือ library ที่เกี่ยวกับ FFT เพื่อให้ผู้จัดทำเข้าใจพื้นฐานของการประมวลผลภาพแบบดิจิทัลได้อย่างลึกซึ้ง

1. Properties of the Fourier Transform

1.1) ทำการแปลง Fourier Transform ของภาพ “[Cross.pgm](#)” (200×200) เนื่องจากในการใช้ FFT จำเป็นต้องให้ภาพมีขนาดที่อยู่ในรูปของ 2^n ดังนั้น อาจต้องทำการ Pad ก่อน และให้แสดงภาพผลลัพธ์ในรูปของ amplitude และ phase spectra’

ขั้นตอนและแนวความคิดการทำ Fourier Transform ของภาพ Cross.pgm

1.อ่านไฟล์ภาพ

อ่านไฟล์ภาพ Cross.pgm เข้าสู่โปรแกรม VSCODE และมี Octave เป็นตัวช่วยในการคำนวณ และแปลงข้อมูลภาพให้อยู่ในรูปแบบตัวเลขชนิด double เพื่อให้สามารถคำนวณเชิงตัวเลขได้(เฉพาะส่วนที่มีการแปลง)

2.แปลงภาพเป็น Grayscale (กรณีอินพุตไม่เป็นGrayscale)

- ตรวจสอบว่าภาพเป็นภาพสีหรือไม่
- หากเป็นภาพสี จะทำการเฉลี่ยค่า RGB เพื่อให้ได้ภาพขาว-ดำ 2 มิติ
- เพื่อให้เหมาะสมกับการแปลงฟูเรียร์

3.แสดงผลภาพต้นฉบับ

- แสดงภาพที่ยังไม่ได้ทำ padding โดย figureออกมาทางการplot สำหรับใช้เปรียบเทียบกับผลลัพธ์ในขั้นตอนถัดไป

4.Zero Padding ภาพ

เนื่องจากการคำนวณ Fourier Transform ในโดเมนดิจิทัลนั้นต้องการ size input ของ ข้อมูลที่เหมาะสม จึงจำเป็นต้องทำการ zero padding โดยขยายภาพเป็นขนาด 256×256 เพื่อป้องกันความบิดเบี้ยวของผลลัพธ์หรือเอาต์พุตที่ได้ และเพิ่มประสิทธิภาพในการคำนวณ FFT เติมค่า 0 (สีดำ) ไปในพื้นที่ที่ได้เพิ่มไปดังกล่าว

5. คำนวณ Fourier Transform

ใช้คำสั่ง `fft2()` เพื่อแปลงภาพจาก spatial domain ไปยัง frequency domain กดใช้คำสั่ง `fftshift()` เพื่อเลื่อนผลความถี่ศูนย์กลางไปยังจุดศูนย์กลางภาพ นั่นคือจะทำให้สามารถวิเคราะห์ผลสเปกตรัมได้ง่ายขึ้น กล่าวคือผลของความถี่ที่รูปภาพได้รับของภาพเอาต์พุต

6. คำนวณ Amplitude Spectrum คำนวณ Phase Spectrum

คำนวณขนาดของแต่ละความถี่ด้วยค่า magnitude ใช้ `log` เพื่อลดช่วงค่าความสว่าง เพื่อให้สามารถแสดงผลได้ชัดเจน คำนวณมุมเฟส θ แต่ละความถี่ของภาพอินพุต ซึ่งแสดงข้อมูลโครงสร้างและตำแหน่งของภาพ

7. แสดงผลลัพธ์

Plot แสดงผลการ padded

Plot ผลของamplitude spectrum

Plot ผลของphase spectrum เพื่อวิเคราะห์คุณสมบัติของภาพในโดเมนความถี่

ผลการทดลอง

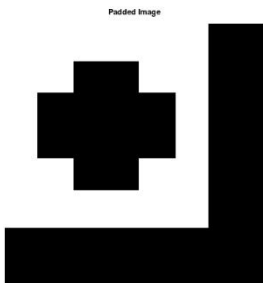
Original Image (Unpadded)



ภาพอินพุต Cross.pgm (Unpadded)

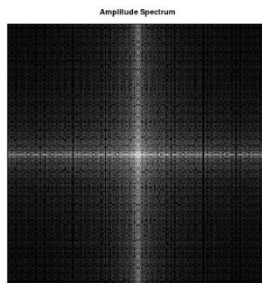
ผลที่ได้จากการpad zero และการ Transform ทาง Phase และ Spectrum

Padded Image



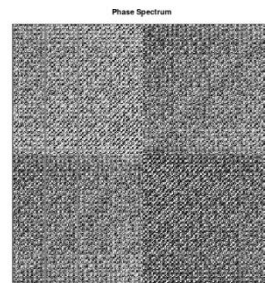
Amplitude Spectrum

ลักษณะเฉพาะเชิงขนาด



Phase

ลักษณะเฉพาะเชิงเฟส



วิเคราะห์ผล

จากผลการแปลง Fourier Transform ของภาพ Cross.pgm แสดงให้เห็นว่าภาพสามารถถูกแยกออกเป็นสเปกตรัมขนาดและเฟสในโดเมนความถี่ได้ หลังจากที่ทำ zero padding นั้นภาพจะถูกขยายขนาดโดยเติมค่า 0 รอบ ๆ เพื่อช่วยในการคำนวณ FFT ไม่เกิดความผิดพลาด และภาพ amplitude spectrum จะแสดงความผลทางความถี่ ซึ่งจะได้ว่าองค์ประกอบความถี่ต่ำอยู่บริเวณกึ่งกลาง ซึ่งแทนโครงสร้างหลักของรูปภาพบาท ส่วนรายละเอียดของขอบภาพจะกระจายออกไปเป็นความถี่สูง ในขณะที่ phase spectrum แสดงข้อมูลตำแหน่งและโครงสร้างของภาพ จะทำให้ได้ว่าการ Fourier transform ช่วยให้เข้าถึงผลของความถี่และตำแหน่งของภาพได้

1.2) ให้ทำการคูณ phase spectrum ที่ได้ในข้อ 1.1 ด้วย complex number ค่าหนึ่ง เพื่อที่ว่าเมื่อทำการ inverse Fourier transform แล้ว ภาพผลลัพธ์ที่ได้ ย้ายด้วยจำนวนในแกน x และ y เป็น (20,30).

ขั้นตอนและแนวคิดการทำงาน

1 .คำนวณ Amplitude และ Phase Spectrum

ทำการคำนวณหา parameters ของ Amplitude spectrum ซึ่งคือขนาดของความถี่ (Spectrum) และต้องหา Phase spectrum

2 .ทำการการเลื่อนภาพ

กำหนดระยะเลื่อนในแกน x และ y (เพราะต้องการการย้ายไปที่ 20,30)
โดยใช้ตัวแปรกำหนดค่าดังนี้

$$x0 = 20$$

$$y0 = 30$$

โดยจะเก็บจำนวนเต็ม $20,30 \in \mathbb{Z}^+$ ไว้ใน $x0,y0$

3. ปรับ phase ด้วย complex exponential (ทำการคูณ)

ใช้สมบัติของ Fourier transform ที่ทำให้เกิดการเลื่อนภาพใน spatial domain
นั่นคือการคูณด้วย phase shift ใน frequency domain แล้วคำนวณ phase ใหม่จากผลที่ได้
(จากCodeในภาคผนวก)

$\text{new_phase} = \text{phase} + \text{shift_phase}$ จะทำการสร้าง spectrum ใหม่ด้วย amplitude เดิม + phase ใหม่
โดยขึ้นกับimage sizeของภาพอินพุต

```
G(u,v) = abs(F_shift(u,v)) * exp(1j*new_phase);
```

4. Inverse Fourier Transform

ใช้คำสั่ง `ifftshift()` คืนตำแหน่งทางความถี่

ใช้คำสั่ง `ifft2()` แปลงกลับเป็น spatial domain

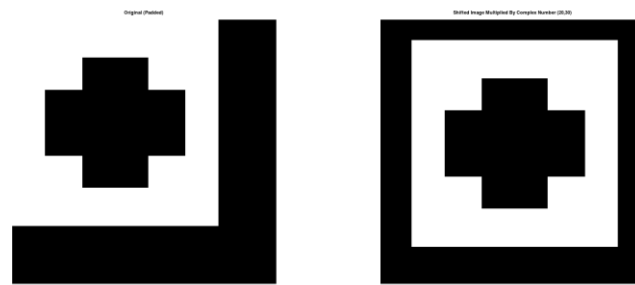
จะได้ผลของภาพที่ถูกเลื่อนตำแหน่ง

5. แสดงผลลัพธ์

ต้องแสดงว่าเปรียบเทียบภาพก่อนเลื่อน

กับภาพที่ถูกเลื่อน (20,30) ต่างกันอย่างไร

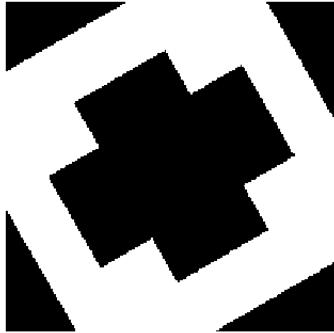
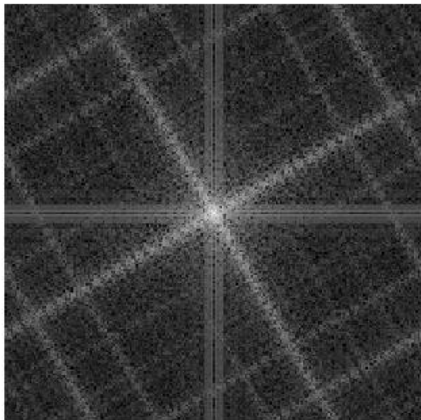
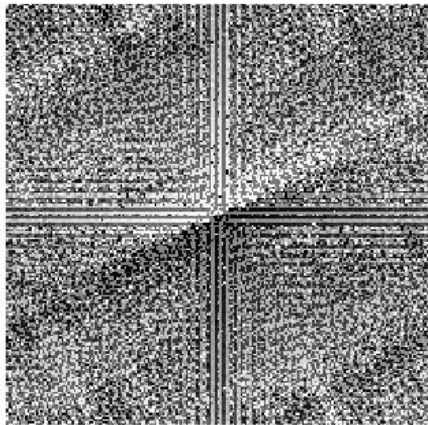
ผลการทดลองที่ได้



วิเคราะห์ผล

การคูณ phase spectrum ด้วย complex number ที่ออกแบบตามสมบัติการเลื่อนของ Fourier transform ทำให้เมื่อทำ inverse transform แล้ว ภาพในโดเมนเชิงพื้นที่ถูก **เลื่อนตำแหน่งตามต้องการ (20,30)** โดย **โครงสร้างและรายละเอียดของภาพยังคงเดิม** ไม่เกิดการบิดเบือน นี่แสดงให้เห็นว่าสมบัติการเลื่อนของฟูเรียร์ช่วยควบคุมตำแหน่งภาพได้อย่างแม่นยำผ่านโดเมนความถี่ โดยไม่ต้องแก้ไขภาพโดยตรงใน spatial domain.

1.3) ทำการหมุนภาพ “[Cross.pgm](#)” ไป 30 องศา และแสดงผลของการแปลงฟูเรียร์ ให้ทำการวิเคราะห์ว่าเกิดอะไรขึ้น

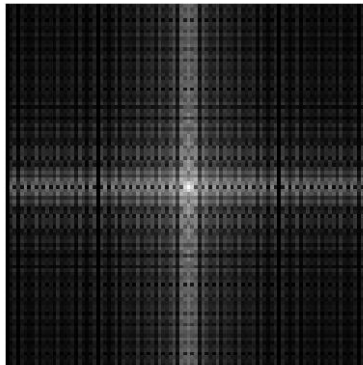
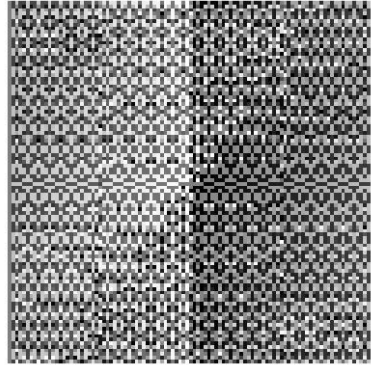
Image	Original (A)	Rotated (B)
Input Image	Original 	Rotated 30 deg 
ผลการแปลงฟูเรียร์ของภาพB	Amplitude Spectrum  Amplitude Spectrum (B)	Phase Spectrum  Phase Spectrum (B)

วิเคราะห์ผล

ผลลัพธ์จากการหมุนภาพ Cross.pgm ไป 30 องศา ได้ว่าการเปลี่ยนตำแหน่งของภาพในโดเมนเชิงพื้นที่จะส่งผลโดเมนความถี่ กล่าวคือหลังจากหมุนภาพ โครงสร้างของรูปกากบาทได้หมุนเอียงไปตามมุมที่กำหนด และเมื่อทำ Fourier Transform จะเห็นว่า amplitude spectrum สามารถหมุนตามไปด้วย ซึ่งคล้ายๆกับความสมบัติที่สำคัญของ Fourier transform ที่กล่าวว่าการหมุนใน spatial domain จะทำให้สเปกตรัมใน frequency domain หมุนในแบบของทางเดียวกัน ส่วน เฟส spectrum ก็สามารถใช้ข้อมูลตำแหน่งและโครงสร้างของภาพ จากผลการแปลงฟูเรียร์และจากสมบัติการหมุน ได้ว่าการหมุนเห็นความสัมพันธ์ระหว่าง spatial domain และ frequency domain ไปทางเดียวกัน

1.4) ทำการ Down-sample “[Cross.pgm](#)” เพื่อให้รูปมีขนาด 100×100 หลังจากนั้นทำการแปลง Fourier Transform ให้แสดงภาพผลลัพธ์ในรูปของ amplitude และ phase spectra ให้ทำการวิเคราะห์ว่าเกิดอะไรขึ้น

ผลการทดลอง

Down-Sampled 100 x 100	Amplitude Spectrum	Phase Spectrum
		

วิเคราะห์ผล อภิปรายผลการทดลองเพิ่มเติม และวิเคราะห์ทฤษฎีที่เกี่ยวข้องเพิ่มเติม

จากการให้ภาพอินพุต Cross.pgm เป็นภาพดิจิทัลที่มีขนาดเท่ากับไฟล์ต้นฉบับ และทำการ down-sample ภาพให้มีขนาด 100×100 แล้ว ต้องแสดงว่าเมื่อลดจำนวนตัวอย่างของภาพส่งผลต่อการกระจายความถี่ของภาพในโดเมนของฟูเรียร์

จาก สมบัติSampling และ สมบัติFourier Transform นั่นคือ การลดจำนวนตัวอย่างในโดเมนภาพให้สเปกตรัมความถี่ถูกบีบและเกิดการซ้ำของความถี่ และจากสมบัติAliasing นั่นคืออัตราการ sampling ต่ำเกินไปจะเกิดการซ้อนทับของความถี่สูง

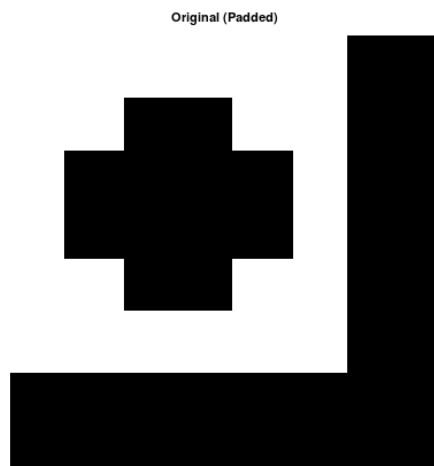
ได้ว่าเมื่อภาพถูก down-sample ความถี่บางส่วนจะซ้อนทับกันในทางความถี่ ทำให้รายละเอียดของภาพลดลง และสเปกตรัมความถี่แสดงการบีบและการสูญเสียข้อมูลบางส่วน

ดังนั้น การ down-sampling ภาพ Cross.pgm ไปเป็น 100×100 ทำให้ข้อมูลรายละเอียดลดลงและเกิด aliasing ซึ่งสอดคล้องกับทฤษฎีการแปลงฟูเรียร์ และ sampling theorem

สรุปผล

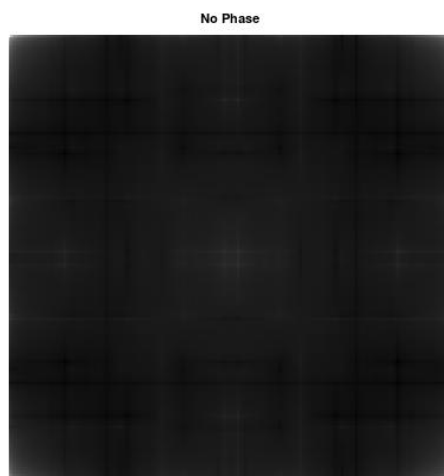
การ down-sample ภาพ Cross.pgm ให้มีขนาด 100×100 ทำให้จำนวนข้อมูลของภาพลดลง ส่งผลให้รายละเอียดบางส่วนหายไป เมื่อพิจารณาในโดเมนฟูเรียร์จะพบว่าสเปกตรัมความถี่เกิดการบีบและมีการซ้อนทับของความถี่ ซึ่งอาจทำให้เกิด aliasing ได้ ดังนั้นภาพที่ได้จะมีความคมชัดลดลงและสูญเสียรายละเอียดบางส่วน แสดงให้เห็นความสัมพันธ์ระหว่างการลดจำนวนตัวอย่างกับการเปลี่ยนแปลงของข้อมูลในโดเมนความถี่อย่างชัดเจน

1.5) ใช้ผลการแปลง inverse Fourier Transform ของผลลัพธ์ที่ได้ในข้อ 1.1



ผลการทดลอง

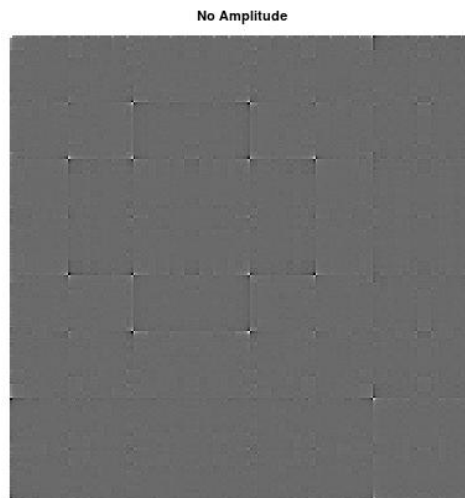
1.5.1) ไม่ใช่ข้อมูล phase



วิเคราะห์ผล

เมื่อทำ inverse Fourier transform โดยใช้เฉพาะ amplitude spectrum และไม่ใช่ข้อมูล phase ภาพที่ได้จะไม่สามารถคงรูปร่างเดิมไว้ได้ แม้ว่าจะยังมีค่าความถี่บางความถี่เหลืออยู่ แต่โครงสร้างและตำแหน่งของวัตถุในภาพจะหายไปโดยรวม ทำให้ภาพดูเบลอหรือกระจาย ไม่สามารถบอกได้ว่าเป็นรูปอะไร ได้ว่าเฟส นั้นเป็นส่วนสำคัญที่เก็บข้อมูลโครงสร้างและตำแหน่งของภาพ

1.5.2) ไม่ใช่ข้อมูล amplitude



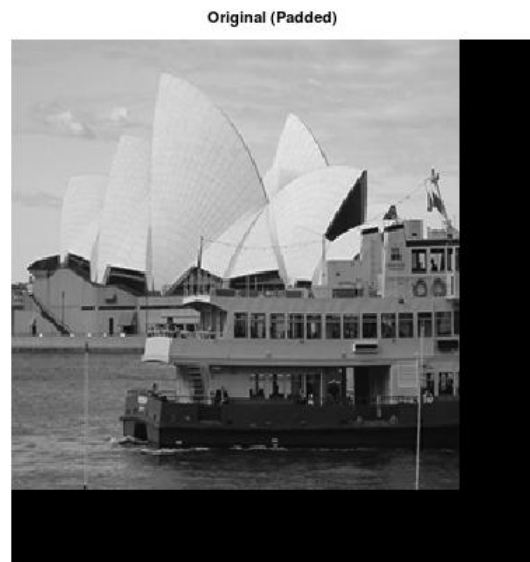
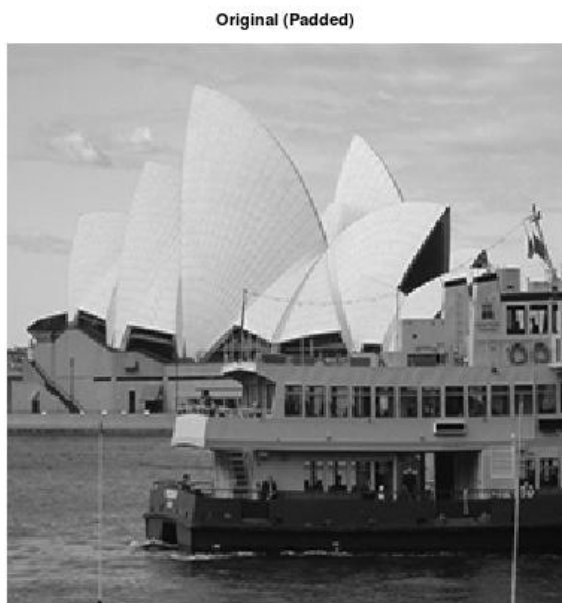
วิเคราะห์ผล

เมื่อทำ inverse Fourier transform โดยใช้เฉพาะ phase spectrum และไม่ใช่ amplitude ภาพที่ได้ยังคงแสดงโครงสร้างหลักและตำแหน่งของวัตถุได้ แต่ว่าความสว่างและรายละเอียดบางส่วนจะผิดเพี้ยนไป แสดงให้เห็นว่า phase มีบทบาทสำคัญต่อรูปร่างและทิศทาง รายละเอียด ภาพมากกว่า amplitude

1.6) ให้ทำการทดลองเช่นเดียวกับข้อ 1.5 ด้วยภาพ “[Operahouse.pgm](#)” (256×256)

ผลการทดลอง

“[Operahouse.pgm](#) Padded (256 x 256)”

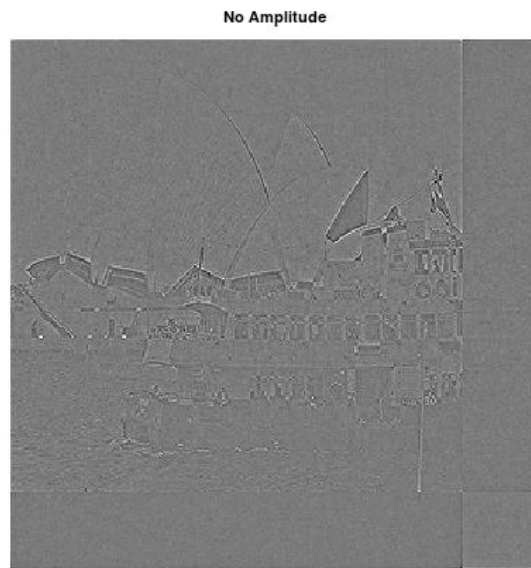


300 x 300

1.5.1) ไม่ใช่ข้อมูล phase



1.5.2) ไม่ใช่ข้อมูล amplitude



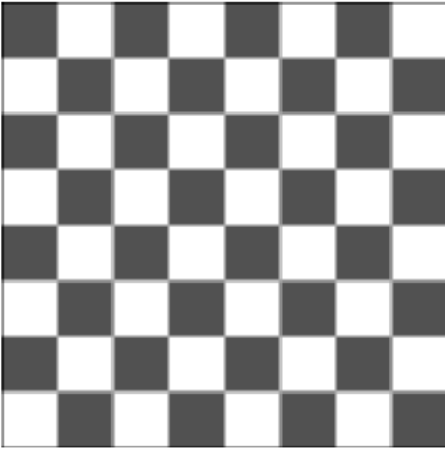
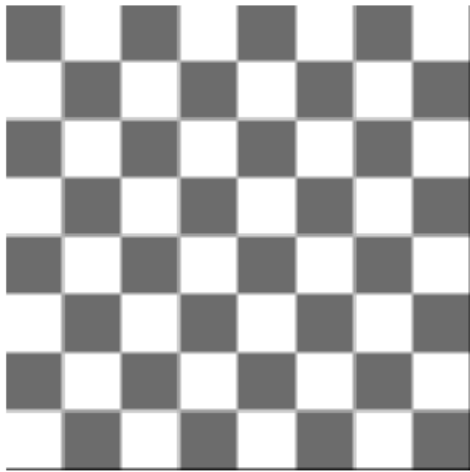
วิเคราะห์ผล

เมื่อทำ inverse Fourier transform โดยใช้เฉพาะ phase spectrum และไม่ใช่ amplitude จะให้ผลคล้ายกับการทดลองในหัวข้อ 1.5 นั่นคือ ภาพที่ได้ยังคงแสดงโครงสร้างหลักและตำแหน่งของวัตถุได้ แต่ว่าความสว่างและรายละเอียดบางส่วนจะผิดเพี้ยนไป แสดงให้เห็นว่า phase มีบทบาทสำคัญต่อรูปร่างและทิศทางรายละเอียด ภาพมากกว่า amplitude

1.7) (a) ทำ convolution ภาพ “[Chess.pgm](#)” (256×256) ด้วย mask หรือ kernel ขนาดเล็กอันหนึ่ง เพื่อทำการ Blur ภาพ และ (b) ให้ทำการ filter ใน frequency domain ด้วย Fourier transform ของ Kernel ที่ใช้ในข้อ (a) เพื่อทำการ Blur ภาพด้วย เปรียบเทียบผลที่ได้ทั้งสองแบบ ว่ามีความแตกต่างหรือหรือเหมือนกันอย่างไร

ผลการทดลอง

ผลการเบลอภาพที่ได้

(a) ทำ convolution ภาพ ด้วย mask หรือ kernel เพื่อเบลอภาพ	(b) ทำการ filter ใน frequency domain ด้วย Fourier transform ของ Kernel ที่ใช้ในข้อ (a) เพื่อเบลอภาพ
Spatial Blur 	Frequency Blur 

จะได้อะไรจากการทำ Blur ด้วย convolution ใน spatial domain และการ filter ใน frequency domain ของภาพ (a) , (b) ให้ผลลัพธ์ที่ใกล้เคียงกัน เนื่องจากทฤษฎี convolution การคูณในโดเมนความถี่มีความสัมพันธ์กับการ convolution ในโดเมนภาพ ได้ว่าทั้งสองวิธีให้ผลการเบลอภาพในลักษณะใกล้เคียงกัน

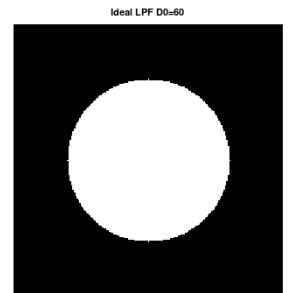
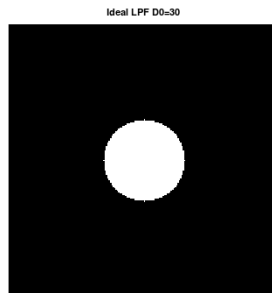
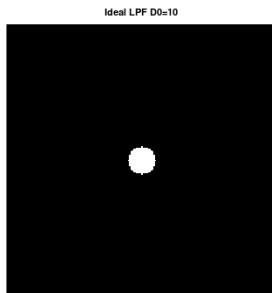
วิเคราะห์ผลและสรุปผลการทดลอง

จากการทดลองเบลอภาพด้วยการทำ convolution ใน spatial domain และการกรองใน frequency domain พบว่าภาพที่ได้ออกมามีลักษณะเกือบเหมือนกัน คือภาพดูนุ่มขึ้น รายละเอียดและขอบที่คมลดลง เนื่องจากทั้งสองวิธีเป็นการลดความถี่สูงของภาพเหมือนกัน ตามหลักการของ Fourier ที่บอกว่าการคูณในโดเมนความถี่ให้ผลเทียบเท่ากับการ convolution ในภาพ จะได้ว่าความแตกต่างที่เห็นมีเพียงเล็กน้อยจากการคำนวณหรือการ padding ดังนั้นสามารถสรุปได้ว่าทั้งสองวิธีให้ผลการเบลอที่สอดคล้องกัน

2. Filter Design

2.1) ให้ใช้ ideal low-pass filter กับภาพ “[Cross.pgm](#)” โดยที่เปลี่ยน cutoff frequency และให้ศึกษา ringing effect ที่เกิดขึ้น หลังจากนั้นให้ทำการทดลองซ้ำด้วย Non-ideal filter อื่น

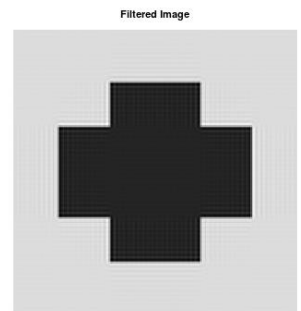
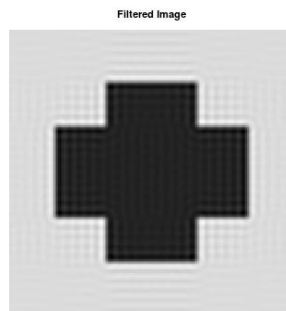
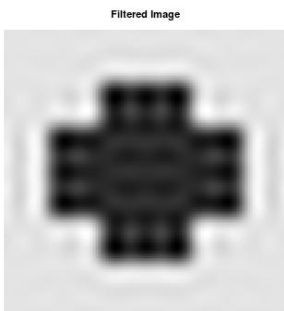
ผลการทดลอง



1. ใช้ Ideal low pass filter ในการกรอง

Gaussian Lowpass Filter ที่ค่า D_0 ที่ต่างกัน จะได้ว่าลักษณะของ filter ในโดเมนความถี่ จะเห็นว่าเมื่อค่า cutoff เพิ่มขึ้น วงกลมสีขาวมีขนาดใหญ่ขึ้น ซึ่งมีการอนุญาตให้ความถี่สูงผ่านได้มากขึ้น ในทางกลับกัน cutoff ต่ำจะตัดความถี่สูงออกเกือบทั้งหมดจะทำได้เป็น low pass filter

ผลที่ได้เมื่อนำไปใช้กับ Cross.pgm

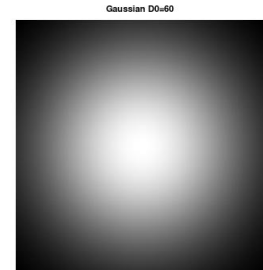
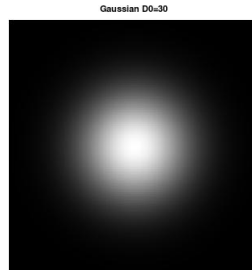
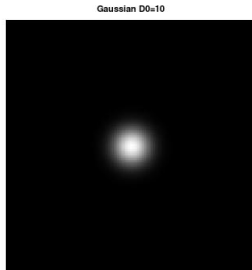


ในแถวล่าง คือภาพหลังผ่านการกรอง เมื่อ cutoff ต่ำ (รูปซ้าย) ภาพจะเบลอมากและมีลักษณะสั้นหรือคลื่นรอบขอบวัตถุ ซึ่งเรียกว่า ringing effect เกิดจากการตัดความถี่แบบฉับพลันของ ideal filter เมื่อเพิ่ม cutoff (รูปกลางและขวา) รายละเอียดภาพเริ่มกลับมาชัดขึ้น และ ringing ลดลง เพราะมีความถี่สูงผ่านได้มากขึ้น

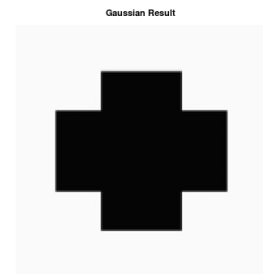
2. ใช้ Gaussian Low Pass Filter (ไม่อุดมคติ) ในการกรอง

ผลการทดลอง

Gaussian Lowpass Filter ที่ค่า D_0 (Cut off) ที่ต่างกัน



ผลของ Gaussian Filter (เอาท์พุท) ของ image



ภาพด้านบน (แถวบนแรก) เป็น Gaussian filter ที่ใช้ค่า cutoff frequency ต่างกัน (ให้ $D_0 = 10, 30, 60$) จะได้ว่ารูป filter ที่ได้นั้น มีลักษณะเป็นการไล่ระดับแบบนุ่ม (smooth transition) ไม่ได้ตัดความถี่แบบฉับพลันเหมือน ideal lowpass filter ก่อนหน้านี้สอดคล้องกับผลตอบสนองทางความถี่ของ Gaussian Low Pass Filter ที่ค่อยๆ ตัดความถี่อย่างช้าๆ นั่นคือไม่ได้ตัดความถี่หรือกรองความถี่บางองค์ประกอบออกทันทีทันใดเหมือน Ideal low pass filter

จะได้ว่า ที่ $D_0 = 10$

filter อนุญาตให้ผ่านเฉพาะความถี่ต่ำมาก ภาพที่ได้จึงเบลอมาก ขอบภาพนุ่ม แต่ ไม่เกิด ringing effect ชัดเจน

ที่ $D_0 = 30$

ความถี่สูงผ่านได้มากขึ้น ภาพเริ่มคมขึ้น รายละเอียดกลับมา แต่ยังคงความนุ่มได้

ที่ $D_0 = 60$

ผ่านความถี่สูงได้มาก ภาพใกล้เคียงต้นฉบับ ขอบคมขึ้น และยังไม่เกิด ringing effect

วิเคราะห์ผลและสรุปผลการทดลอง

จากการทดลองนี้คือการเปรียบเทียบการใช้ Ideal low-pass filter และ Gaussian low-pass filter พบว่า Ideal filter ทำการตัดความถี่สูงอย่างฉับพลัน ส่งผลให้ภาพที่ได้เกิดการเบลอพร้อมทั้งปรากฏ ringing effect บริเวณขอบวัตถุอย่างมาก เนื่องจากการเปลี่ยนแปลงความถี่ที่ไม่ต่อเนื่อง ในขณะที่ Gaussian filter ลดความถี่สูงแบบค่อยเป็นค่อยไป (ความนุ่มนั้นมีค่อนข้างสูงมาก) ทำให้ภาพที่ได้มีความเบลอนุ่มนวลกว่าและขอบภาพเรียบกว่า โดยแทบไม่เกิด ringing effect ดังนั้น Gaussian filter จึงให้ผลลัพธ์ที่เหมาะสมกับงานประมวลผลภาพมากกว่าเมื่อพิจารณาด้านคุณภาพของภาพหลังการกรอง

2.2) ให้ใช้ low-pass filter หลายแบบ โดยที่เปลี่ยนค่าตัวแปรต่างๆ รวมถึงขนาดของ filter ด้วย กับภาพ “[Chess_noise.pgm](#)” และ “[Operahouse_noise.pgm](#)” เพื่อลด noise และให้เปรียบเทียบผลกับ Median filter ที่มีการเปลี่ยนขนาด และให้ใช้ RMS ในการคำนวณหาความแตกต่างระหว่างผลลัพธ์ที่ได้กับ ภาพที่ไม่มี Noise “[Chess.pgm](#)” and “[Operahouse.pgm](#)”

ผลการทดลอง

คำนวณค่า rms

```
=== Chess Results ===
Ideal LP cutoff=20 RMS=26.13
Gaussian LP cutoff=20 RMS=20.72
Ideal LP cutoff=40 RMS=17.67
Gaussian LP cutoff=40 RMS=15.05
Ideal LP cutoff=80 RMS=16.78
Gaussian LP cutoff=80 RMS=14.88

Median Filter Chess:
Median size=3 RMS=12.58
Median size=5 RMS=12.42
Median size=7 RMS=14.20

=== Opera Results ===
Ideal LP cutoff=20 RMS=17.49
Gaussian LP cutoff=20 RMS=15.45
Ideal LP cutoff=40 RMS=14.55
Gaussian LP cutoff=40 RMS=12.22
Ideal LP cutoff=80 RMS=14.95
Gaussian LP cutoff=80 RMS=14.24

Median Filter Opera:
Median size=3 RMS=13.49
Median size=5 RMS=14.43
Median size=7 RMS=15.99
```

โดยที่ค่า rms ที่คำนวณในที่นี้คือค่าความแตกต่างของผลลัพธ์ระหว่างค่าที่มีnoise และไม่มีnoise

วิเคราะห์ผล

จากการเปรียบเทียบค่า RMS ว่าเป็น Gaussian low-pass filter ให้ผลลัพธ์ดีกว่า Ideal filter ในทุกๆ cutoff นั่นคือมีการเปลี่ยนผ่านความถี่อย่างต่อเนื่อง ringing effect จะเกิดน้อยลง และรายละเอียดภาพชัดขึ้น ในส่วนของภาพ Chess median filter ขนาด 3x3 และ 5x5 ให้ค่า RMS ต่ำที่สุด นั่นคือมีประสิทธิภาพสูงในการลด impulse noise ขณะที่ภาพ Operahouse Gaussian low-pass filter ที่ cutoff ปานกลางให้เลขจากการคำนวณได้ค่อนข้างที่จะดี จะได้ว่าลักษณะ noise เหมือนมีการกระจายในโดเมนความถี่มากกว่า โดยรวมค่า RMS เป็นการวัดค่าที่ได้ชัดเจน

ภาคผนวก

ข้อที่ 1

clear;

clc;

% --- อ่านภาพ ---

img = imread("Cross.pgm");

img = double(img);

% แปลง grayscale ถ้าเป็น RGB

if ndims(img) == 3

 img = mean(img,3);

end

[M,N] = size(img);

figure;

imshow(img,[]);

title("Original Image (Unpadded)");

% --- pad ---

P = 256;

Q = 256;

padded = zeros(P,Q);

for i = 1:M

for j = 1:N

 padded(i,j) = img(i,j);

end

end

% --- FFT ---

F = fft2(padded);

F_shift = fftshift(F);

% --- amplitude ---

amp = zeros(P,Q);

for i = 1:P

```
    for j = 1:Q
        amp(i,j) = log(1 + abs(F_shift(i,j)));
    end
end
```

```
% --- phase ---
phase = zeros(P,Q);
for i = 1:P
    for j = 1:Q
        phase(i,j) = angle(F_shift(i,j));
    end
end
figure;
```

```
subplot(1,3,1);
imshow(padded,[]);
title("Padded Image");
```

```
subplot(1,3,2);
imshow(amp,[]);
title("Amplitude Spectrum");
```

```
subplot(1,3,3);
imshow(phase,[]);
title("Phase Spectrum");
```

```
pause;
```

ข้อที่2

clear;

clc;

% --- อ่านภาพ ---

img = imread("Cross.pgm");

img = double(img);

% แปลง grayscale ถ้าเป็น RGB

if ndims(img) == 3

 img = mean(img,3);

end

[M,N] = size(img);

% --- แสดง original ---

figure;

imshow(img,[]);

title("Original Image (Unpadded)");

% --- pad ---

P = 256;

Q = 256;

padded = zeros(P,Q);

for i = 1:M

for j = 1:N

 padded(i,j) = img(i,j);

end

end

% --- FFT ---

F = fft2(padded);

F_shift = fftshift(F);

% --- amplitude ---

amp = zeros(P,Q);

```

for i = 1:P
    for j = 1:Q
        amp(i,j) = log(1 + abs(F_shift(i,j)));
    end
end

% --- phase ---
phase = zeros(P,Q);
for i = 1:P
    for j = 1:Q
        phase(i,j) = angle(F_shift(i,j));
    end
end

% --- shift amount ---
x0 = 20;
y0 = 30;

% --- สร้าง spectrum ใหม่จาก amplitude + phase ที่ถูกเลื่อน ---
G = zeros(P,Q);

for u = 1:P
    for v = 1:Q

        shift_phase = -2*pi*((u-1)*x0/P + (v-1)*y0/Q);

        new_phase = phase(u,v) + shift_phase;

        G(u,v) = abs(F_shift(u,v)) * exp(1j*new_phase);

    end
end

% --- inverse shift กลับ ---
G_unshift = ifftshift(G);

% --- inverse FFT ---
shifted_img = real(ifft2(G_unshift));

```

```
figure;
```

```
subplot(1,2,1);
```

```
imshow(padded,[]);
```

```
title("Original (Padded)");
```

```
subplot(1,2,2);
```

```
imshow(shifted_img,[]);
```

```
title("Shifted Image Multiplied By Complex Number (20,30)");
```

```
pause
```

ข้อ 3

clear;

clc;

% --- อ่านภาพ ---

img = imread("Cross.pgm");

img = double(img);

[M,N] = size(img);

% --- หมุน 30 องศา ---

theta = 30*pi/180;

cx = M/2;

cy = N/2;

rot = zeros(M,N);

for x = 1:M

for y = 1:N

% shift center

xt = x - cx;

yt = y - cy;

% inverse rotation

xr = xt*cos(theta) + yt*sin(theta);

yr = -xt*sin(theta) + yt*cos(theta);

% shift back

xr = round(xr + cx);

yr = round(yr + cy);

if xr>=1 && xr<=M && yr>=1 && yr<=N

rot(xr,yr) = img(x,y);

end

end

end

% --- FFT ---

F = fftshift(fft2(rot));

amp = log(1 + abs(F));

phase = angle(F);

% --- แสดงผล ---

figure;

subplot(2,2,1);

imshow(img,[]);

title("Original");

subplot(2,2,2);

imshow(rot,[]);

title("Rotated 30 deg");

subplot(2,2,3);

imshow(amp,[]);

title("Amplitude Spectrum");

subplot(2,2,4);

imshow(phase,[]);

title("Phase Spectrum");

pause;

ข้อที่4

clear;

clc;

%Down sample image

% --- อ่านภาพ ---

img = imread("Cross.pgm");

img = double(img);

[M,N] = size(img);

% --- Down-sample 2:1 ---

Md = M/2;

Nd = N/2;

down = zeros(Md,Nd);

for i = 1:Md

for j = 1:Nd

down(i,j) = img(2*i-1, 2*j-1);

end

end

% --- FFT ---

F = fftshift(fft2(down));

amp = log(1 + abs(F));

phase = angle(F);

% --- แสดงผล ---

figure;

subplot(1,3,1);

imshow(down,[]);

title("Down-sampled 100x100");

subplot(1,3,2);

imshow(amp,[]);

title("Amplitude Spectrum");

```
subplot(1,3,3);  
imshow(phase,[]);  
title("Phase Spectrum");
```

```
pause;
```

```
ข้อที่5
```

```
clear;  
clc;
```

```
% --- อ่านภาพ ---
```

```
img = imread("OperaHousePGM_256_256.pgm");  
img = double(img);
```

```
[M,N] = size(img);
```

```
% --- pad ---
```

```
%P = 256;
```

```
%Q = 256;
```

```
P = 300
```

```
Q = 300
```

```
padded = zeros(P,Q);
```

```
for i = 1:M
```

```
    for j = 1:N
```

```
        padded(i,j) = img(i,j);
```

```
    end
```

```
end
```

```
% --- FFT ---
```

```
F = fftshift(fft2(padded));
```

```
mag = abs(F);
```

```
phase = angle(F);
```

```
%% =====
```

```

% 1.5.1 ไม่มี phase
%% =====

F_no_phase = mag; % phase = 0

img_no_phase = real(ifft2(ifftshift(F_no_phase)));

%% =====
% 1.5.2 ไม่มี amplitude
%% =====

F_no_mag = zeros(P,Q);

for i = 1:P
    for j = 1:Q
        F_no_mag(i,j) = exp(1j*phase(i,j));
    end
end

img_no_mag = real(ifft2(ifftshift(F_no_mag)));

%% --- แสดงผล ---
figure;

subplot(1,3,1);
imshow(padded,[]);
title("Original (Padded)");

subplot(1,3,2);
imshow(img_no_phase,[]);
title("No Phase");

subplot(1,3,3);
imshow(img_no_mag,[]);
title("No Amplitude");

pause;

```

ข้อที่6

```
% =====  
% Convolution theorem verification  
% spatial blur = frequency blur  
% =====
```

```
clear;  
clc;
```

```
%% =====  
% อ่านภาพ  
%% =====
```

```
img = imread("Chess.pgm");  
img = double(img);
```

```
[M,N] = size(img);
```

```
%% =====  
% kernel blur 3x3  
%% =====
```

```
kernel = ones(3,3)/9;
```

```
pad = 1;
```

```
%% =====  
% (A) SPATIAL CONVOLUTION  
%% =====
```

```
pimg = zeros(M+2*pad,N+2*pad);
```

```
% padding  
for i=1:M  
    for j=1:N  
        pimg(i+pad,j+pad) = img(i,j);  
    end  
end
```

```
blur_spatial = zeros(M,N);
```

```
for i=1:M
    for j=1:N
        sumv = 0;
        for u=1:3
            for v=1:3
                sumv += kernel(u,v)*pimg(i+u-1,j+v-1);
            end
        end
        blur_spatial(i,j) = sumv;
    end
end
```

```
%% =====
% (B) FREQUENCY CONVOLUTION
%% =====
```

```
% linear convolution size
```

```
P = M + 2;
```

```
Q = N + 2;
```

```
%% =====
%% =====
```

```
P = M + 2;
```

```
Q = N + 2;
```

```
img_pad = zeros(P,Q);
img_pad(1:M,1:N) = img;
```

```
H = zeros(P,Q);
H(1:3,1:3) = kernel;
```

```
% shift center kernel
H = circshift(H, [-1 -1]);
```

```

F = fft2(img_pad);
Hf = fft2(H);

G = F .* Hf;

conv_full = real(ifft2(G));

blur_freq = conv_full(2:M+1,2:N+1);

%% =====
% difference check
%% =====

diff_img = blur_spatial - blur_freq;

%% =====
% plot
%% =====

figure;

subplot(2,2,1);
imshow(img,[]);
title("Original");

subplot(2,2,2);
imshow(blur_spatial,[]);
title("Spatial Blur");

subplot(2,2,3);
imshow(blur_freq,[]);
title("Frequency Blur");

pause;

```

ข้อที่ 2.1

clear;

clc;

close all;

%% --- อ่านภาพ ---

img = imread("Cross.pgm");

img = double(img);

[M,N] = size(img);

%% --- FFT ---

F = fftshift(fft2(img));

[u,v] = meshgrid(-N/2:N/2-1, -M/2:M/2-1);

D = sqrt(u.^2 + v.^2);

%% =====

% IDEAL LOW PASS FILTER

%% =====

cutoff = [10 30 60];

figure;

for k = 1:3

 D0 = cutoff(k);

 H = double(D <= D0);

 G = F .* H;

 img_lpf = real(ifft2(ifftshift(G)));

 subplot(2,3,k);

 imshow(H,[]);

 title(["Ideal LPF D0=",num2str(D0)]);

```

subplot(2,3,k+3);
imshow(img_lpf,[]);
title("Filtered Image");
end

%% =====
% NON-IDEAL (Gaussian LPF)
%% =====

figure;
for k = 1:3

    D0 = cutoff(k);

    H = exp(-(D.^2)/(2*(D0^2)));

    G = F .* H;

    img_gauss = real(ifft2(ifftshift(G)));

    subplot(2,3,k);
    imshow(H,[]);
    title(["Gaussian D0=",num2str(D0)]);

    subplot(2,3,k+3);
    imshow(img_gauss,[]);
    title("Gaussian Result");
end
pause

```


ข้อที่ 2.2

pkg load image

% ----- Utility -----

```
function out = rms_error(a,b)
    a = double(a);
    b = double(b);
    out = sqrt(mean((a(:)-b(:)).^2));
end
```

```
function H = ideal_lp(M,N,D0)
    [u,v] = meshgrid(-N/2:N/2-1, -M/2:M/2-1);
    D = sqrt(u.^2 + v.^2);
    H = double(D <= D0);
end
```

```
function H = gaussian_lp(M,N,D0)
    [u,v] = meshgrid(-N/2:N/2-1, -M/2:M/2-1);
    D = sqrt(u.^2 + v.^2);
    H = exp(-(D.^2)/(2*(D0^2)));
end
```

```
function out = apply_fft_filter(img,H)
    F = fftshift(fft2(img));
    G = H .* F;
    out = real(ifft2(ifftshift(G)));
end
```

% ----- Load images -----

```
chess_clean = imread("Chess.pgm");
chess_noise = imread("Chess_noise.pgm");
```

```
opera_clean = imread("OperaHousePGM_256_256.pgm");
opera_noise = imread("OperaHousePGM_256_256_noise.pgm");
```

% convert to double

```
chess_clean = double(chess_clean);
chess_noise = double(chess_noise);
```

```

opera_clean = double(opera_clean);
opera_noise = double(opera_noise);

[M,N] = size(chess_noise);

cutoffs = [20 40 80];

printf("\n=== Chess Results ===\n");

for D0 = cutoffs

    % Ideal LP
    H = ideal_lp(M,N,D0);
    out = apply_fft_filter(chess_noise,H);
    err = rms_error(out,chess_clean);
    printf("Ideal LP cutoff=%d RMS=%.2f\n",D0,err);

    % Gaussian LP
    H = gaussian_lp(M,N,D0);
    out = apply_fft_filter(chess_noise,H);
    err = rms_error(out,chess_clean);
    printf("Gaussian LP cutoff=%d RMS=%.2f\n",D0,err);

end

% Median filters
printf("\nMedian Filter Chess:\n");
for k=[3 5 7]
    out = medfilt2(chess_noise,[k k]);
    err = rms_error(out,chess_clean);
    printf("Median size=%d RMS=%.2f\n",k,err);
end

% ----- Opera -----
[M,N] = size(opera_noise);

printf("\n=== Opera Results ===\n");

```

for D0 = cutoffs

```
H = ideal_lp(M,N,D0);  
out = apply_fft_filter(opera_noise,H);  
err = rms_error(out,opera_clean);  
printf("Ideal LP cutoff=%d RMS=%.2f\n",D0,err);
```

```
H = gaussian_lp(M,N,D0);  
out = apply_fft_filter(opera_noise,H);  
err = rms_error(out,opera_clean);  
printf("Gaussian LP cutoff=%d RMS=%.2f\n",D0,err);
```

end

```
printf("\nMedian Filter Opera:\n");
```

for k=[3 5 7]

```
out = medfilt2(opera_noise,[k k]);  
err = rms_error(out,opera_clean);  
printf("Median size=%d RMS=%.2f\n",k,err);
```

end